```
void interruptible_wait(std::condition_variable& cv,
                        std::unique_lock<std::mutex>& lk)
{
    interruption_point();
    this_thread_interrupt_flag.set_condition_variable(cv);        ❶
    cv.wait(lk);                                                  ❷
    this_thread_interrupt_flag.clear_condition_variable();        ❸
    interruption_point();
}
```

Assuming the presence of some functions for setting and clearing an association of a condition variable with an interrupt flag, this code is nice and simple. It checks for interruption, associates the condition variable with the `interrupt_flag` for the current thread ❶, waits on the condition variable ❷, clears the association with the condition variable ❸, and checks for interruption again. If the thread is interrupted during the wait on the condition variable, the interrupting thread will broadcast the condition variable and wake you from the wait, so you can check for interruption. Unfortunately, this code is *broken*: there are two problems with it. The first problem is relatively obvious if you have your exception safety hat on: `std::condition_variable::wait()` can throw an exception, so you might exit the function without removing the association of the interrupt flag with the condition variable. This is easily fixed with a structure that removes the association in its destructor.

The second, less-obvious problem is that there's a race condition. If the thread is interrupted after the initial call to `interruption_point()`, but before the call to `wait()`, then it doesn't matter whether the condition variable has been associated with the interrupt flag, because *the thread isn't waiting and so can't be woken by a notify on the condition variable*. You need to ensure that the thread can't be notified between the last check for interruption and the call to `wait()`. Without delving into the internals of `std::condition_variable`, you have only one way of doing that: use the mutex held by `lk` to protect this too, which requires passing it in on the call to `set_condition_variable()`. Unfortunately, this creates its own problems: you'd be passing a reference to a mutex whose lifetime you don't know to another thread (the thread doing the interrupting) for that thread to lock (in the call to `interrupt()`), without knowing whether